

zyo-zxrh-ihy

Recursion

def fact(n):

if n == 1:

return n

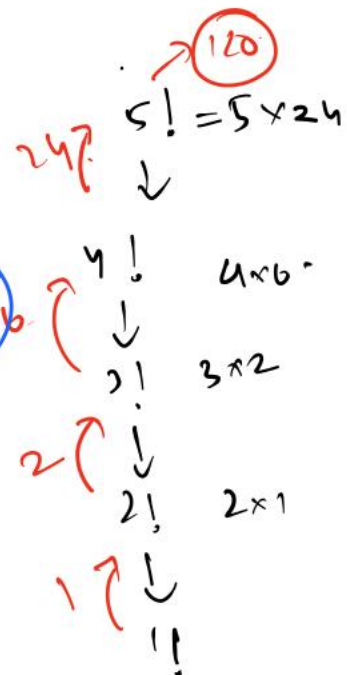
smallAns = fact(n-1)

return n * smallAns

I.S

B.C

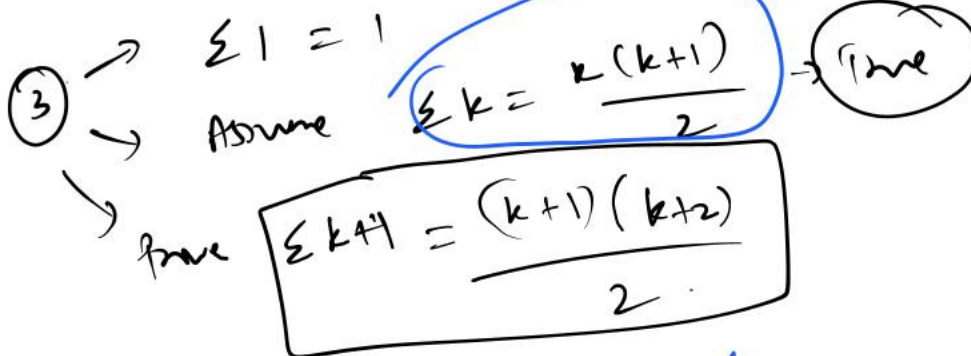
I.H



PMI → Principle of Mathematical Induction.

$$\sum n = \frac{n(n+1)}{2}$$

$$1 \leq k \leq n$$



$$\begin{aligned} \sum k+1 &= \sum k + (k+1) \\ &= \frac{k(k+1)}{2} + (k+1) \end{aligned}$$

$$k(k+1) + 2(k+1)$$

$$\Sigma \frac{(k+1)}{2} (k+2)$$

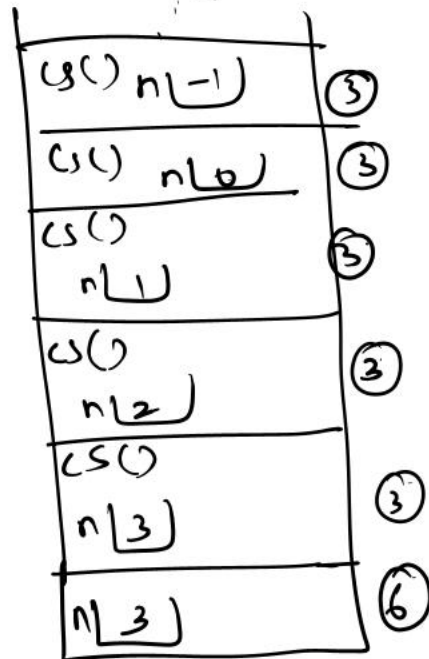
```

1  def callStack(n) :
2      print(n)
3      callStack(n - 1)
4
5  n = 3
6  callStack(n)

```

3 2 1 0 -1

console



0	1	2	3	4
10	20	40	30	25

elem | 25 | o/p: 4

If element is present → once → return index.
 → multiple times → return first occurrence

Not present → return -1

0	1	2	3	4	5
10	20	40	25	30	20

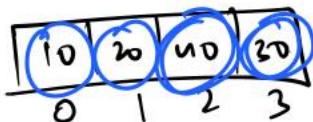
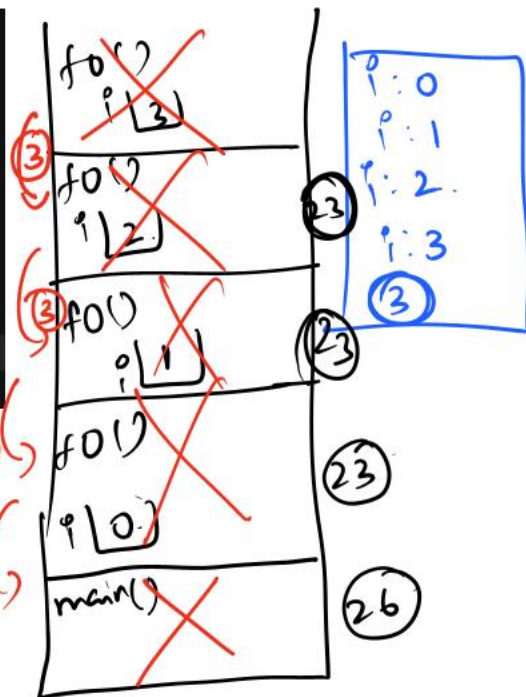
elem | 20 | i | 0



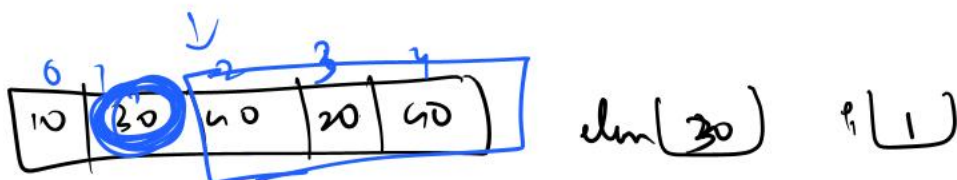
```

17 def firstOccurrence(nums, element, i):
18     if i == len(nums):
19         return -1
20     print("i is : ", i)
21     if nums[i] == element:
22         return i
23     return firstOccurrence(nums, element, i + 1)
24
25 nums = [10, 20, 40, 30]
26 print(firstOccurrence(nums, 30, 0))

```



`elem [30]`



[

1	2	3
---	---	---

nums ↑

matrix

	0	1	2
0	1	2	3
1	4	5	10
2	3	7	12

10	20	40
----	----	----

→ sums → link string

20	30
----	----

["10", "20", "40"] → [10, 20, 40]

30	40	70	100	120	25
----	----	----	-----	-----	----

↓
append to
matrix

1672. Richest Customer Wealth

Solved ✓

Easy Topics Companies Hint

You are given an $m \times n$ integer grid `accounts` where `accounts[i][j]` is the amount of money the i^{th} customer has in the j^{th} bank. Return the **wealth** that the richest customer has.

A customer's **wealth** is the amount of money they have in all their bank accounts. The richest customer is the customer that has the maximum **wealth**.

Example 1:

Input: `accounts = [[1,2,3],[3,2,1]]`

Output: 6

Explanation:

1st customer has wealth = $1 + 2 + 3 = 6$ 2nd customer has wealth = $3 + 2 + 1 = 6$

	0	1
0	1	5
1	7	3
2	3	2

o/p: 10

```
class Solution:
    def maximumWealth(self, accounts: List[List[int]]) -> int:
        output = 0
        for i in range(len(accounts)) :
            current = 0
            for j in range(len(accounts[i])) :
                current += accounts[i][j]
            output = max(current, output)
        return output
```

→ P.N.O: 1672

→ T.C → $O(m \times n)$

→ S.C → $O(1)$

```
class Solution:
    def maximumWealth(self, accounts: List[List[int]]) -> int:
        output = 0
        for row in accounts :
            current = sum(row)
            output = max(current, output)
        return output
```

→ $m \times n$

766. Toeplitz Matrix

Solved ✓

Easy

Topics

Companies

Hint

Given an $m \times n$ matrix, return `true` if the matrix is Toeplitz. Otherwise, return `false`.

A matrix is **Toeplitz** if every diagonal from top-left to bottom-right has the same elements.

	0	1	2	3
0	1	2	3	4
1	5	1	2	3
2	9	5	1	2

$[0,0] \leftrightarrow [1,1]$

$[0,1] \leftrightarrow [1,2]$

$[0,2] \leftrightarrow [1,3]$

$[1,0] \leftrightarrow [2,1]$

$[1,1] \leftrightarrow [2,2]$

$[1,2] \leftrightarrow [2,3]$

$[i,j] \leftrightarrow [i+1,j+1]$

$i \rightarrow 0 \text{ to } 1$
 $j \rightarrow 0 \text{ to } 2$


```

class Solution:
    def isToeplitzMatrix(self, matrix: List[List[int]]) ->
    bool:
        for i in range(len(matrix) - 1) :
            for j in range(len(matrix[i]) - 1) :
                if matrix[i][j] != matrix[i + 1][j + 1] :
                    return False
        return True

```

↓
p.no: 766

↓
T.C → $O(m \times n)$

↓
S.C → $O(1)$

999. Available Captures for Rook

Solved ✓

Easy

Topics

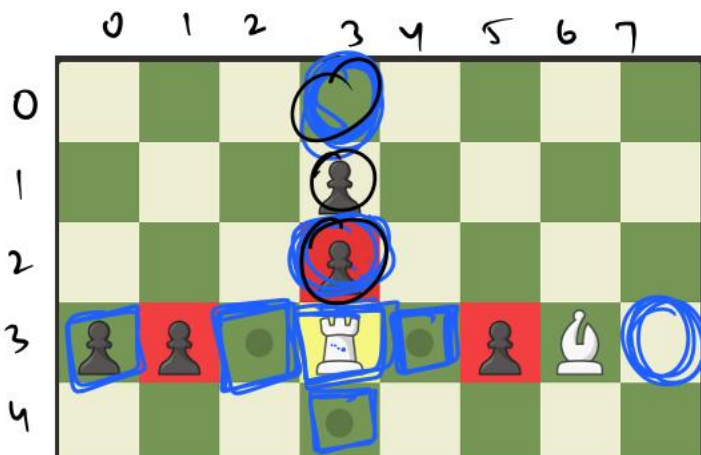
Companies

You are given an 8×8 matrix representing a chessboard. There is **exactly one** white rook represented by 'R'. some number of white bishops 'B' and some number of black pawns 'p'. Empty squares are represented by '.'.

A rook can move any number of squares horizontally or vertically (up, down, left, right) until it reaches another piece or the edge of the board. A rook is **attacking** a pawn if it can move to the pawn's square in one move.

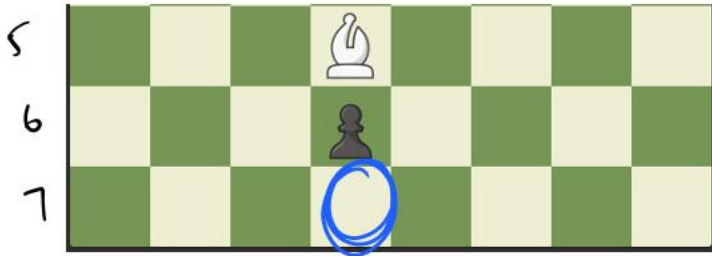
Note: A rook cannot move through other pieces, such as bishops or pawns. This means a rook cannot attack a pawn if there is another piece blocking the path.

Return the **number of pawns** the white rook is **attacking**.



Rook position → (3,3)

Direction	Common	Start	End
up	3	0-1	0
down	3	1-7	7
		0-1	1



Right i $j+1$ 1
 Left i $j-1$ 0

```
def numRookCaptures(self, board: List[List[str]]) -> int:
    pawns = 0
    for i in range(8):
        for j in range(8):
            if board[i][j] == 'R':
                # Up -> j is common, i - 1 -> 0
                for k in range(i - 1, -1, -1):
                    if board[k][j] == 'B':
                        break
                    elif board[k][j] == 'p':
                        pawns += 1
                        break
                # Right -> i is common, j + 1, 7
                for k in range(j + 1, 8):
                    if board[i][k] == 'B':
                        break
                    elif board[i][k] == 'p':
                        pawns += 1
                        break
```

```
# Left -> i is common, j - 1, 0
for k in range(j - 1, -1, -1):
    if board[i][k] == 'B':
        break
    elif board[i][k] == 'p':
        pawns += 1
        break
# Down -> j is common, i + 1 -> 7
for k in range(i + 1, 8):
    if board[k][j] == 'B':
        break
    elif board[k][j] == 'p':
        pawns += 1
        break
return pawns
```

$\downarrow$
 $P.No \rightarrow (999)$

$\downarrow$
 $S.C \rightarrow 0(1)$

$\downarrow$
 $T.C \rightarrow 0(1)$

202. Happy Number

Easy Topics Companies

Write an algorithm to determine if a number n is happy.

A **happy number** is a number defined by the following process:

- Starting with any positive integer, replace the number by the sum of the squares of its digits.
- Repeat the process until the number equals 1 (where it will stay), or it loops endlessly in a cycle which does not include 1.
- Those numbers for which this process ends in 1 are happy.

Return `true` if n is a happy number, and `false` if not.

Example 1:

Input: $n = 19$
 Output: `true`

$19 \rightarrow (1^2 + 9^2)$

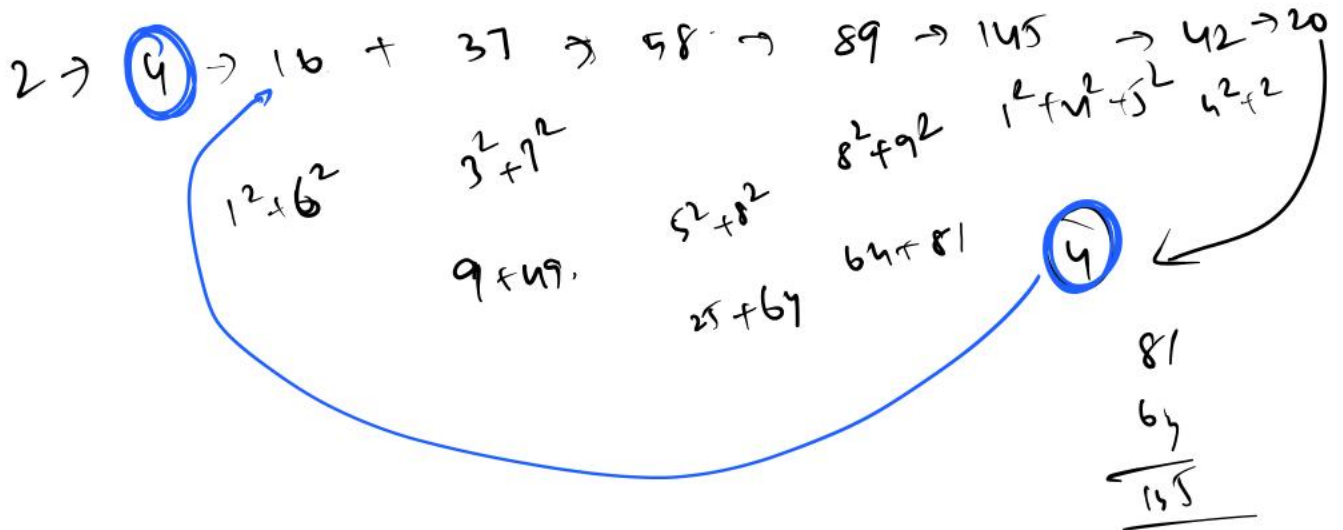
$82 \rightarrow 64 + 4$

$68 \rightarrow 36 + 64$

$100 \rightarrow 1$

$19 \rightarrow 1$
 n

total $\left[\begin{smallmatrix} 8 \times 8^2 \\ 0 \end{smallmatrix} \right]$ digit $\left[\begin{smallmatrix} 1 \\ 9 \end{smallmatrix} \right]$



```
class Solution:
    def isHappy(self, n: int) -> bool:
        seen = set()
        while n != 1:
            if n in seen:
                return False
            seen.add(n)
            total = 0
            while n != 0:
                digit = n % 10
                total += (digit * digit)
                n //= 10
            n = total
        return True
```

